

在 GKE 部署開放模型

來源：<https://codelabs.developers.google.com/codelabs/production-ready-ai-with-gc/5-deploying-agents/deploying-open-models-gke?hl=zh-tw>

步驟數：7

「瞭解如何使用 Ollama 在 Google Kubernetes Engine (GKE) 上部署 Gemma 等開放式模型，以提供高輸送量服務。」

目錄

1. [簡介](#)
2. [專案設定](#)
3. [開啟 Cloud Shell 編輯器](#)
4. [使用 Ollama 執行 Gemma](#)
5. [在 GKE Autopilot 上使用 Ollama 部署 Gemma](#)
6. [清除](#)
7. [結論](#)

1. 簡介

總覽

本實驗室的目標是提供實務經驗，讓您在 Google Cloud 上部署開放模型，從簡單的本機設定，逐步進展到 [Google Kubernetes Engine \(GKE\)](#) 的正式版部署作業。您將瞭解如何使用適合開發生命週期各階段的工具。

本實驗室的流程如下：

- **快速原型設計：**您會先在本機執行 Ollama 模型，瞭解如何輕鬆上手。
- **正式版部署：**最後，您會使用 Ollama 做為可擴充的服務引擎，將模型部署至 GKE Autopilot。

瞭解開放式模型

「開放模型」一詞現在通常是指**所有人都能下載及使用的生成式機器學習模型**。也就是說，模型的架構，以及最重要的訓練參數或「權重」，都會公開發布。

相較於通常只能透過限制性 API 存取的封閉模型，這項透明度具有下列優勢：

- **深入分析：**開發人員和研究人員可以「深入瞭解」模型，掌握其內部運作方式。
- **自訂：**使用者可以透過微調程序，讓模型適應特定工作。
- **創新：**讓社群以現有強大模型為基礎，建構創新應用程式。

注意：請注意，雖然有時會將這些模型稱為「開放原始碼模型」，但這並不一定準確。真正的開放原始碼版本通常會包含訓練程式碼，但開放模型往往並非如此。此外，許多開放模型都有使用限制的授權。

Google 的貢獻和 Gemma 系列

多年來，Google 一直是開放原始碼 AI 運動的基礎貢獻者。2017 年發表的論文「[導入注意力機制就對了](#)」中，介紹了革命性的 Transformer 架構，這也是幾乎所有現代大型語言模型的基礎。隨後，BERT、T5 和經過指令微調的 Flan-T5 等具里程碑意義的開放模型相繼問世，不斷突破極限，為全球的研究和開發工作注入活力。

Google 秉持開放創新的悠久歷史，推出 [Gemma](#) 系列模型。Gemma 模型採用與強大封閉來源 Gemini 模型相同的研究成果和技術，但提供開放權重。對於 Google Cloud 客戶而言，這項服務結合了尖端技術和開放原始碼的彈性，可讓他們控管模型生命週期、整合多元生態系統，以及採用多雲端策略。

Gemma 3 焦點

在本實驗室中，我們將著重於 Gemma 3，這是這個系列最新且功能最強大的模型。Gemma 3 模型輕巧但先進，可在單一 GPU 甚至 CPU 上有效率地執行。

- **效能和大小：**Gemma 3 模型輕巧但先進，可在單一 GPU 甚至 CPU 上有效率地執行。以同類產品來說，這些模型可提供優異的品質和最先進的效能。
- **模態：**這些模型是多模態模型，可處理文字和圖片輸入內容，並生成文字輸出內容。
- **主要功能：**Gemma 3 具有 12.8 萬個詞元的大型脈絡窗口，並支援超過 140 種語言。
- **用途：**這些模型非常適合用於各種工作，包括回答問題、重點摘要和推論。

重要術語

使用開放式模型時，您會遇到幾個常見字詞：

- **預先訓練**：使用大量多元資料集訓練模型，學習一般語言模式。這些模型基本上就是強大的自動完成機器。
- **指令微調**會微調預先訓練模型，使其更擅長遵循特定指令和提示。這些模型「知道如何對話」。
- **模型變體**：開放式模型通常會發布多種大小 (例如 Gemma 3 有 1B、4B、12B 和 27B 參數版本)，以及變體，例如經過指令微調 (-it)、預先訓練或量化以提高效率。
- **資源需求**：大型語言模型體積龐大，需要大量運算資源才能託管。雖然這些模型可以在本機執行，但部署至雲端可帶來顯著價值，特別是使用 Ollama 等工具進行最佳化，以提升效能和可擴充性時。

為什麼要使用 GKE 提供開放模型？

本實驗室會引導您從簡單的本機模型執行作業，到在 Google Kubernetes Engine (GKE) 上進行大規模的正式版部署。Ollama 等工具非常適合快速製作原型，但實際工作環境有一系列嚴格要求，而 GKE 正好能滿足這些需求。

對於大規模 AI 應用程式，您需要的不是只有執行中的模型，而是彈性、可擴充且有效率的服務基礎架構。GKE 提供這項基礎。以下說明選擇 GKE 的時機和原因：

- **透過 Autopilot 簡化管理作業**：GKE Autopilot 會為您管理底層基礎架構。您只需專注於應用程式設定，Autopilot 就會自動佈建及調整節點。
- **高效能和擴充性**：透過 GKE 的自動調整資源配置功能，處理要求嚴苛的變動流量。確保應用程式能以低延遲時間提供高處理量，並視需要擴大或縮減規模。
- **大規模提升成本效益**：有效管理資源。GKE 可將工作負載縮減至零，避免支付閒置資源的費用，您也可以運用 Spot VM 大幅降低無狀態推論工作負載的成本。
- **可攜性與豐富的生態系統**：透過可攜式 Kubernetes 部署作業，避免受制於特定廠商。GKE 也提供廣大的雲端原生 (CNCF) 生態系統存取權，讓您使用一流的監控、記錄和安全工具。

簡而言之，當 AI 應用程式準備好投入生產，且需要專為大規模、高效能和成熟運作而建構的平台時，您就會改用 GKE。

課程內容

在本實驗室中，您將瞭解如何執行下列工作：

- 使用 Ollama 在本機執行開放模型。
 - 使用 Ollama 將開放模型部署至 [Google Kubernetes Engine \(GKE\)](#) Autopilot，以提供服務。
 - 瞭解如何從本機開發架構，逐步建構 GKE 上的正式環境等級服務架構。
-

步驟 2 · 約 0 分鐘

2. 專案設定

Google 帳戶

如果沒有個人 Google 帳戶，請[建立 Google 帳戶](#)。

請使用個人帳戶，而非公司或學校帳戶。

公司和學校帳戶可能設有限制，導致您無法啟用本實驗室所需的 API。

登入 Google Cloud 控制台

使用個人 Google 帳戶登入 [Google Cloud 控制台](#)。

啟用計費功能

設定個人帳單帳戶

如果使用 Google Cloud 抵免額設定計費，可以略過這個步驟。

如要設定個人帳單帳戶，請[前往這裡在 Cloud 控制台中啟用帳單](#)。

注意事項：

- 完成本實驗室的 Cloud 資源費用應不到 \$1 美元。
- 您可以按照本實驗室結尾的步驟刪除資源，以免產生後續費用。
- 新使用者可獲得價值 [\\$300 美元的免費試用期](#)。

建立專案 (選用)

如果沒有要用於本實驗室的現有專案，請[在這裡建立新專案](#)。

步驟 3 · 約 0 分鐘

3. 開啟 Cloud Shell 編輯器

1. 按一下這個連結，直接前往 [Cloud Shell 編輯器](#)
2. 如果系統在今天任何時間提示您授權，請點選「授權」繼續操作。

Authorize Cloud Shell

gcloud is requesting your credentials to make a GCP API call.

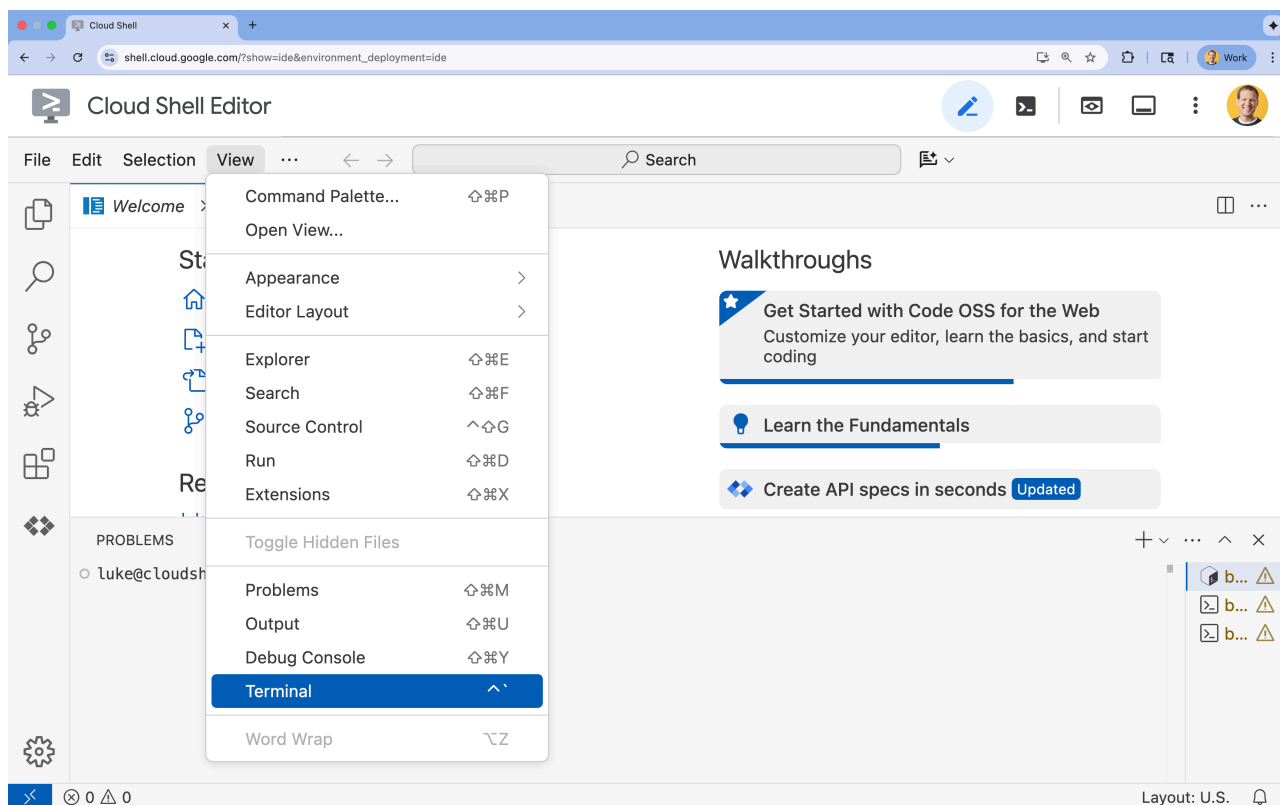
Click to authorize this and future calls that require your credentials.

REJECT

AUTHORIZE

3. 如果畫面底部未顯示終端機，請開啟終端機：

- 按一下「查看」
- 按一下「終端機」



4. 在終端機中，使用下列指令設定專案：

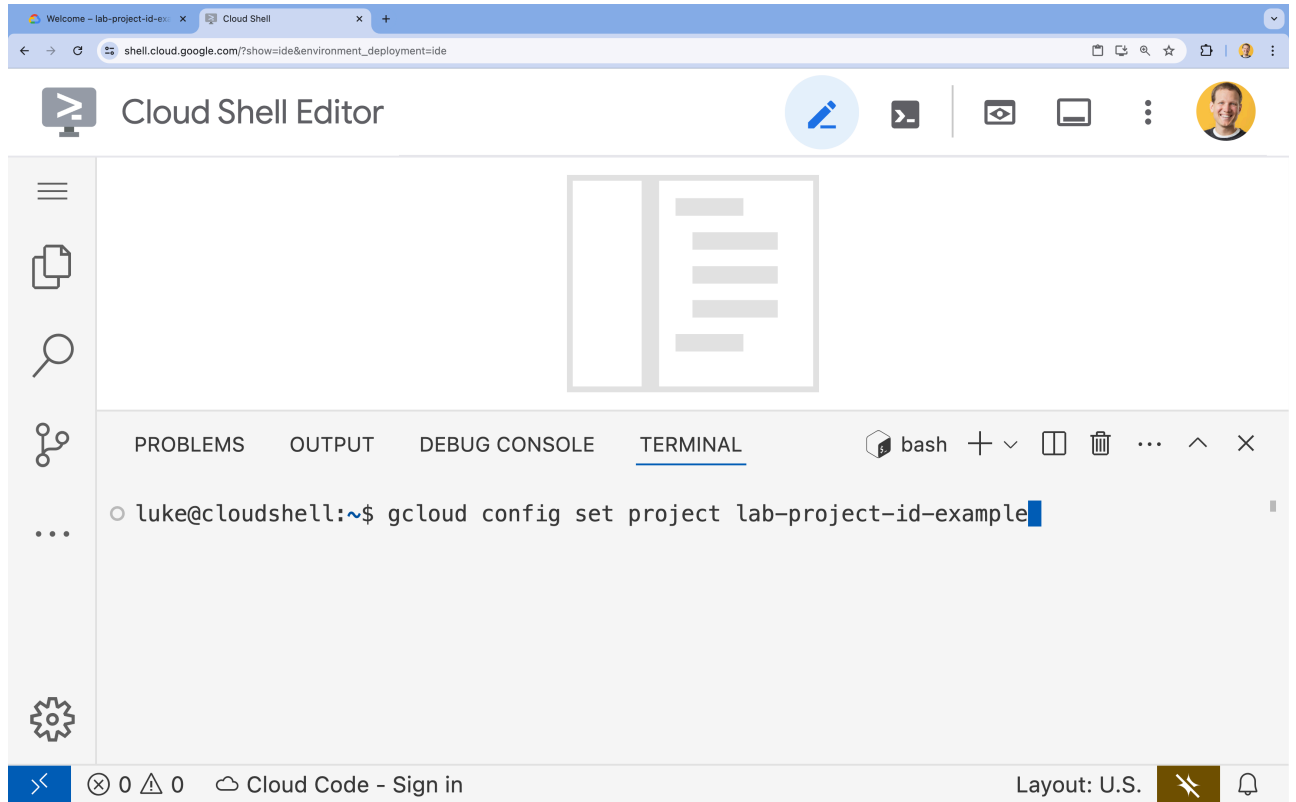
```
gcloud config set project [PROJECT_ID]
```

- 範例：

```
gcloud config set project lab-project-id-example
```

- 如果忘記專案 ID，可以使用下列指令列出所有專案 ID：

```
gcloud projects list
```



5. 您應會看到下列訊息：

```
Updated property [core/project].
```

如果看到 **WARNING** 並收到 **Do you want to continue (Y/n)?** 提示，表示您可能輸入了錯誤的專案 ID。按下 **n** 和 **Enter**，然後再次嘗試執行 **gcloud config set project** 指令。

4. 使用 Ollama 執行 Gemma

您的首要目標是在開發環境中盡快執行 Gemma 3。您將使用 [Ollama](#)，這項工具可大幅簡化在本機執行大型語言模型的方式。這項工作會說明最簡單的方法，讓您開始試用開放模型。

[Ollama](#) 是免費的開放原始碼工具，可讓使用者在自己的電腦上，在本機執行生成模型 (大型語言模型、視覺語言模型等)。這項功能可簡化存取及與這些模型互動的程序，讓使用者更容易使用，並能私下與模型互動。

安裝及執行 Ollama

現在可以安裝 Ollama、下載 Gemma 3 模型，並透過指令列與模型互動。

1. 在 Cloud Shell 終端機中，下載並安裝 Ollama：

```
curl -fsSL https://ollama.com/install.sh | sh
```

這項指令會下載並安裝 Ollama，然後啟動 Ollama 服務。

2. 在背景啟動 Ollama 服務：

```
ollama serve &
```

3. 使用 Ollama 提取 (下載) Gemma 3 1B 模型：

```
ollama pull gemma3:1b
```

注意：下載模型一律需要時間。大型語言模型非常龐大，從幾 GB 到幾百 GB 不等。

4. 在本機執行模型：

```
ollama run gemma3:1b
```

`ollama run` 指令會顯示提示 (`>>>`)，供您向模型提問。

5. 向模型提問，例如，輸入 `why is the sky blue?` 並按下 Enter 鍵。畫面會顯示類似以下的回應：


```
>>> Why is the sky blue?
Okay, let's break down why the sky is blue – it's a fascinating phenomenon related
to how light interacts with the Earth's atmosphere.
Here's the explanation:

**1. Sunlight and Colors:**

* Sunlight appears white, but it's actually made up of all the colors of the
rainbow (red, orange, yellow, green, blue, indigo, and violet).
Think of a prism splitting sunlight.

**2. Rayleigh Scattering:**

* As sunlight enters the Earth's atmosphere...
...
```

6. 如要退出終端機中的 Ollama 提示，請輸入 `/bye` 並按下 Enter 鍵。

搭配 Ollama 使用 OpenAI SDK

Ollama 服務現已啟動，您可以透過程式輔助方式與其互動。您將使用與 Ollama 公開的 API 相容的 OpenAI Python SDK。

1. 在 Cloud Shell 終端機中，使用 `uv` 建立並啟動虛擬環境。確保專案依附元件不會與系統 Python 發生衝突。

```
uv venv --python 3.14
source .venv/bin/activate
```

注意：如果 Cloud Shell 工作階段重新整理，或是您開啟新的終端機分頁，請執行 `source .venv/bin/activate` 重新啟用虛擬環境。

2. 在終端機中安裝 OpenAI SDK：

```
uv pip install openai
```

3. 在終端機中輸入下列指令，建立名為 `ollama_chat.py` 的新檔案：

```
cloudshell edit ollama_chat.py
```

4. 將下列 Python 程式碼貼入 `ollama_chat.py`。這段程式碼會將要求傳送至本機 Ollama 伺服器。

```
from openai import OpenAI

client = OpenAI(
    base_url = 'http://localhost:11434/v1',
    api_key='ollama', # required by OpenAI SDK, but not used by Ollama
)

response = client.chat.completions.create(
    model="gemma3:1b",
    messages=[
        {
            "role": "user",
            "content": "Why is the sky blue?"
        },
    ],
)
print(response.choices[0].message.content)
```

5. 在終端機中執行指令碼：

```
python3 ollama_chat.py
```

幾秒後，您會看到類似於從指令列收到的回覆。

6. 如要試用串流模式，請在終端機中執行下列指令，建立另一個名為 `ollama_stream.py` 的檔案：

```
cloudshell edit ollama_stream.py
```

7. 將下列內容貼到 `ollama_stream.py` 檔案中。請注意要求中的 `stream=True` 參數。模型生成權杖後，就會立即傳回。

```
from openai import OpenAI

client = OpenAI(
    base_url = 'http://localhost:11434/v1',
    api_key='ollama',
)

stream = client.chat.completions.create(
    model="gemma3:1b",
    messages=[
        {
            "role": "user",
            "content": "Why is the sky blue?"
        },
    ],
    stream=True
)
for chunk in stream:
    if chunk.choices[0].delta.content is not None:
        print(chunk.choices[0].delta.content, end="", flush=True)
print()
```

8. 在終端機中執行串流指令碼：

```
python3 ollama_stream.py
```

現在回應會逐一顯示權杖。

串流功能有助於在聊天機器人等互動式應用程式中，打造良好的使用者體驗。串流功能會在生成回覆時逐一顯示回應權杖，使用者不必等待系統生成完整回覆。這能提供即時意見回饋，讓應用程式的反應更靈敏。

所學內容：使用 Ollama 執行開放式模型

您已使用 Ollama 成功執行開放式模型。您已瞭解下載 Gemma 3 等強大模型並與之互動有多簡單，無論是透過指令列介面，還是以 Python 程式輔助，都能輕鬆完成。這個工作流程非常適合快速原型設計和本機開發。您現在已具備紮實的基礎，可以探索更多進階部署選項。

5. 在 GKE Autopilot 上使用 Ollama 部署 Gemma

對於需要簡化作業和擴充性的生產環境工作負載，[Google Kubernetes Engine \(GKE\)](#) 是理想的平台。在這項工作中，您將使用 GKE Autopilot 叢集上的 Ollama 部署 Gemma。

GKE Autopilot 是 GKE 的作業模式，可讓 Google 管理叢集設定，包括節點、資源調度、安全性和其他預先設定。這項服務可打造真正的「無伺服器」Kubernetes 體驗，非常適合執行推論工作負載，完全不必管理底層運算基礎架構。

準備 GKE 環境

在部署至 Kubernetes 的最後一項工作中，您將佈建 GKE Autopilot 叢集。

1. 在 Cloud Shell 終端機中，為專案和所需區域設定環境變數。

```
export PROJECT_ID=$(gcloud config get-value project)
export REGION=europe-west1

gcloud config set compute/region $REGION
```

2. 在終端機中執行下列指令，為專案啟用 GKE API：

```
gcloud services enable container.googleapis.com
```

3. 在終端機中執行下列指令，建立 GKE Autopilot 叢集：

注意：建立叢集約需 10 分鐘。Autopilot 會自動處理節點佈建作業，因此您不需要建立個別的節點集區。

```
gcloud container clusters create-auto gemma-cluster \
  --region $REGION \
  --release-channel rapid
```

4. 在終端機中執行下列指令，取得新叢集的憑證：

```
gcloud container clusters get-credentials gemma-cluster \
  --region $REGION
```

部署 Ollama 和 Gemma

現在您已擁有 GKE Autopilot 叢集，可以部署 Ollama 伺服器。Autopilot 會根據您在部署資訊清單中定義的需求，自動佈建運算資源 (CPU 和記憶體)。

1. 在終端機中執行下列指令，建立名為 `gemma-deployment.yaml` 的新檔案：

```
cloudshell edit gemma-deployment.yaml
```

2. 將下列 YAML 設定貼到 `gemma-deployment.yaml` 中。這項設定定義的部署作業會使用官方 Ollama 映像檔，在 CPU 上執行。

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: ollama-gemma
spec:
  replicas: 1
  selector:
    matchLabels:
      app: ollama-gemma
  template:
    metadata:
      labels:
        app: ollama-gemma
    spec:
      containers:
      - name: ollama-gemma-container
        image: ollama/ollama:0.12.10
        resources:
          requests:
            cpu: "8"
            memory: "8Gi"
            ephemeral-storage: "10Gi"
          limits:
            cpu: "8"
            memory: "8Gi"
            ephemeral-storage: "10Gi"
        # We use a script to start the server and pull the model
        command: ["/bin/bash", "-c"]
        args:
        - |
          ollama serve &
          OLLAMA_PID=$!
          echo "Waiting for Ollama server to start..."
          sleep 5
          echo "Pulling Gemma model..."
          ollama pull gemma3:1b
          echo "Model pulled. Ready to serve."
          wait $OLLAMA_PID
      ports:
      - containerPort: 11434
      env:
      - name: OLLAMA_HOST
        value: "0.0.0.0"
---
apiVersion: v1
kind: Service
metadata:
  name: llm-service
spec:
  selector:
    app: ollama-gemma
  type: ClusterIP

```

```
ports:
- protocol: TCP
  port: 8000
  targetPort: 11434
```

以下說明 Autopilot 的設定：

- **image: ollama/ollama:latest**：指定官方 Ollama Docker 映像檔。
- **resources**：我們明確要求 8 個 vCPU 和 8Gi 的記憶體。GKE Autopilot 會使用這些值佈建基礎運算資源。由於我們未使用 GPU，模型會在 CPU 上執行。8Gi 的記憶體足以容納 Gemma 1B 模型及其脈絡。
- **command/args**：我們覆寫啟動指令，確保 Pod 啟動時會提取模型。這個指令碼會在背景啟動伺服器、等待伺服器就緒、提取 `gemma3:1b` 模型，然後讓伺服器保持運作。
- **OLLAMA_HOST**：將此值設為 `0.0.0.0`，可確保 Ollama 監聽容器內的所有網路介面，讓 Kubernetes 服務能夠存取。

3. 在終端機中，將 Deployment 資訊清單套用至叢集：

```
kubectl apply -f gemma-deployment.yaml
```

Autopilot 佈建資源和啟動 Pod 需要幾分鐘的時間。你可以使用下列方式監控：

```
kubectl get pods --watch
```

等待 Pod 狀態為 `Running`，且 `READY` 為 `1/1`，再繼續操作。

測試 GKE 端點

Ollama 服務現已在 GKE Autopilot 叢集上執行。如要從 Cloud Shell 終端機進行測試，請使用 `kubectl port-forward`。

1. 開啟新的 **Cloud Shell** 終端機分頁 (點選終端機視窗中的「+」圖示)。`port-forward` 指令是封鎖程序，因此需要專屬的終端機工作階段。
2. 在新終端機中執行下列指令，將本機通訊埠 (例如 `8000`) 轉送至服務的通訊埠 (`8000`)：

```
kubectl port-forward service/llm-service 8000:8000
```

畫面會顯示輸出內容，指出轉送作業已開始。讓這個終端機保持執行狀態。

3. 返回原始終端機。
4. 將要求傳送到本機通訊埠 `8000`。Ollama 伺服器會公開與 OpenAI 相容的 API，由於已轉送通訊埠，您現在可以在 `http://127.0.0.1:8000` 存取該 API。

```
curl http://127.0.0.1:8000/v1/chat/completions \  
-H "Content-Type: application/json" \  
-d '{  
  "model": "gemma3:1b",  
  "messages": [  
    {"role": "user", "content": "Explain why the sky is blue."}  
  ]  
'
```

服務會傳回 JSON 回應，其中包含模型的完成內容。

步驟 6 · 約 0 分鐘

6. 清除

如要避免系統向您的 Google Cloud 帳戶收取本實驗室所用資源的費用，請按照下列步驟刪除 GKE 叢集。

1. 在 Cloud Shell 終端機中，刪除 GKE Autopilot 叢集：

```
gcloud container clusters delete gemma-cluster \
  --region $REGION --quiet
```

這個指令會移除叢集和所有相關聯的資源。

7. 結論

做得好！在本實驗室中，您已瞭解在 Google Cloud 上部署開放模型的幾種重要方法。您一開始使用 Ollama 進行本機開發，體驗了簡單快速的作業流程。最後，您使用 Google Kubernetes Engine Autopilot 和 Ollama 架構，將 Gemma 部署至可擴充的正式環境。

您現在已具備相關知識，可透過 Google Kubernetes Engine 部署開放模型，處理需要大量資源且可擴充的工作負載，不必管理基礎架構。

重點回顧

在本實驗室中，您已學會：

- 瞭解開放模型是什麼，以及為何重要。
- 如何透過 Ollama 在本機執行開放模型。
- 瞭解如何使用 [Ollama](#) 進行推論，在 [Google Kubernetes Engine \(GKE\) Autopilot](#) 上部署開放模型。

瞭解詳情

- 若要進一步瞭解 Gemma 模型，請參閱[官方說明文件](#)。
 - 前往 GitHub 上的 [Google Cloud 生成式 AI 存放區](#)，查看更多範例。
 - 進一步瞭解 [GKE Autopilot](#)。
 - 瀏覽 [Vertex AI Model Garden](#)，查看其他可用的開放模型和專屬模型。
-